Java

📘 Theory

- **Collections Framework** → Predefined data structures in Java for storing and manipulating groups of objects.
- **ArrayList**
  - Resizable array implementation.
  - Allows duplicates, maintains insertion order.
  - Common methods: add(), remove(), get(), set(), contains(), size().
- **HashSet**
  - Stores unique elements, no duplicates.
  - No guaranteed order.
- **TreeSet**
  - Stores unique elements in **sorted order**.
  - Implements NavigableSet.
- **HashMap**
  - Stores key-value pairs.
  - Keys are unique, values can be duplicated.
  - Common methods: put(), get(), containsKey(), containsValue(), remove().
- **LinkedList**
  - Doubly linked list implementation.
  - Efficient insertions/deletions compared to ArrayList.
  - Methods: addFirst(), addLast(), offer(), peek(), poll(), pop().

📝 Coding Round Questions

- ArrayList → Write a program to store integers in an ArrayList, update an element, and print all elements.
- HashSet → Demonstrate how HashSet avoids duplicates when adding elements.
- TreeSet → Insert integers into a TreeSet and print them in ascending order.
- HashMap → Count frequency of characters in a string using HashMap.

- LinkedList → Create a LinkedList of names, add elements at the beginning and end, and print them.
- Collections methods → Demonstrate contains(), size(), and lastIndexOf() using ArrayList.

⚡ **Machine Coding Round Challenge**

Problem: Student Management Using Collections

Requirements:

1. Use an **ArrayList** to store student names.
2. Use a **HashSet** to ensure no duplicate student names are added.
3. Use a **TreeSet** to maintain students in sorted order.
4. Use a **HashMap** to store student roll numbers as keys and names as values.
5. Use a **LinkedList** to maintain a queue of students waiting for counseling.
6. Implement methods:
    - addStudent(int roll, String name) → adds student to all collections.
    - removeStudent(int roll) → removes student from collections.
    - displayAll() → prints all students in sorted order.
    - searchStudent(String name) → checks if student exists.

```java
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.TreeSet;

public class Collections {

    public static void main(String[] args) {
        // System.out.println("Collections");

        // framework  => predefined structures

        ArrayList<Integer> names = new ArrayList<>();

        names.add(1);
        names.add(3);
        names.add(2);
        names.add(3);
        // // names.remove(0);
        // names.set(0, 100);
```

```java
// System.out.println(names.get(0));

// System.out.println(names.contains(1000));

// int arr[] = {1,2,3,4,5};

// arr[2]  = 40;


HashSet<String> hs = new HashSet<>();

hs.add("Ravi");

hs.add("Ravi");
System.out.println(hs);

TreeSet<Integer> ts = new TreeSet<>();

ts.add(10);
ts.add(5);
ts.add(1);
ts.add(100);

System.out.println(ts);

HashMap<Integer , String> hm = new HashMap<>();


// hm.put(1, "Harini");
// hm.put(2, "Subhiksha");
// hm.put(3, "manisha");

// System.out.println(hm.values());



// harini =>
//     h - 1
//     a - 1
//     r - 1
//     i - 2
//     n - 1

HashMap<Character , Integer> freq = new HashMap<>();

String name = "harini";

// h a r i n i
```

```
        for(char c : name.toCharArray()){

            freq.put(c, freq.getOrDefault(c, 0 ) + 1);

            System.out.println(c);

        }









        // System.out.println(arr.length);


        // names.remove(2);
        // // names.clear();
        // System.out.println(names);
        // System.out.println(names.size());

        // System.out.println(names.lastIndexOf(3));

        // LinkedList<String> ll = new LinkedList<>();

        // ll.offer("Subhiksha");
        // ll.addFirst("Harini");
        // ll.add("Manisha");


        // System.out.println(ll.peek());

        // System.out.println(ll.poll());
    //    String linked = ll.pop();

    //    System.out.println(linked);

    //     System.out.println(ll);

        // int -> primitive
        // Integer -> non-primitive

        // arraylist
        // linkedlist
    }

}
```